Student notes — Session 6 · Loop patterns, tracing and practice

Slides: [`Class_06_Loop_patterns_and_practice.pdf`](../slides/Class_06_Loop_patterns_and_practice.pdf). These notes are the notes pane of the student copy of the deck.

Slide 1 — Loop patterns in C

The programs of this class are on Blackboard: Week 3 module, *Class 6 code*. It is a zip file. Unzip it and open the folder in Visual Studio Code. Every file has a comment at the top: what the program does, how to run it, and a list called **Try** with small changes to make. Make them. Breaking a program on purpose is a good way to learn what each line does.

How to run a program. Same as every week. In the terminal type `gcc -Wall name.c -o name` and press Enter, then `./name` (or `.\name` in Windows PowerShell) and press Enter.

If a program never stops: click in the terminal and press **Ctrl+C**. You saw this on Tuesday.

On Tuesday you learned how a loop is written. Today is about what loops are used for.

Slide 2 — What Tuesday gave us

What you need from Tuesday:

- A loop has four parts: the **start**, the **check** (the condition), the **body**, and the **update**.
- `for (int i = 1; i <= 5; i++)` makes `i` go 1, 2, 3, 4, 5.
- `i++` means `i = i + 1`. `sum += i` means `sum = sum + i`.
- A loop that never stops: press Ctrl+C.

Slide 3 — Recap: what is on the screen?

Try to answer before you look at the next slide. The loop starts at 5. It goes on while `i` is at least 1. After every round, `i--` takes one away.

Slide 4 — Recap: what is on the screen?

The screen shows `5 4 3 2 1`. To count down, all three parts change: the start is the big number, the check is `>=`, the update is `--`.

Slide 5 — Today's roadmap

Six steps today:

1. Keeping a total, and counting things.
2. Asking again until the input is good.
3. Choosing where a loop starts and stops.
4. `break` and `continue`.
5. Two whole programs, and the bugs everybody makes.
6. Tracing a loop with a pen, and writing your own.

Number 6 is the quiz skill. **Quiz 2 is on Friday 2 October** and has a loop to trace.

Slide 6 — Accumulator pattern

Part 1: building a total, one round at a time.

Slide 7 — Accumulator pattern: building a total

You want to add 1 + 2 + 3 + 4 + 5. You need one box that remembers **the total so far**. Think of the till in a shop. It starts at zero. Every item adds to it. At the end you read the total. In programming this box is called an **accumulator**. It is just a running total.

Slide 8 — The whole program: sum.c

This is the file `01_sum_1_to_5.c`. The `printf` inside the loop is only there so that you can watch the total grow.

The screen shows:

```
after 1: sum = 1
after 2: sum = 3
after 3: sum = 6
after 4: sum = 10
after 5: sum = 15
Sum = 15
```

Slide 9 — Before the loop: start at zero

The first place of the pattern. **Before** the loop: make the box and set it to zero. `int sum = 0;` This line runs once.

Slide 10 — Inside the loop: add to it

The second place. **Inside** the loop: add to the total. `sum += i;` means `sum = sum + i;`. This line runs in every round.

Slide 11 — After the loop: use it

The third place. **After** the loop: the total is ready. Print it, or calculate with it.

Slide 12 — Round 1: the total grows

Round 1: `sum` is 0 and `i` is 1. 0 + 1 = 1. Now `sum` is 1.

Slide 13 — Round 2: the total remembers

Round 2: `sum` is **still 1**. It kept its value from round 1, because the box was made before the loop. 1 + 2 = 3.

Slide 14 — Round 5: the total is ready

The whole trace:

Round	`i`	`sum` before	`sum += i`	`sum` after
1	1	0	0 + 1	1
2	2	1	1 + 2	3
3	3	3	3 + 3	6
4	4	6	6 + 4	10
5	5	10	10 + 5	15

Then `i` becomes 6. `6 <= 5` is false. The loop ends. The line after the loop prints `Sum = 15`.

Slide 15 — Run it

The output is the same as the table. If your screen shows something else, compare your file with `01_sum_1_to_5.c`, line by line.

Slide 16 — The pattern has three places

Learn it by heart.

1. **Before** the loop: make the box and set it to zero. `int sum = 0;`
2. **Inside** the loop: add to it. `sum += i;`
3. **After** the loop: use it. `printf("Sum = %d\n", sum);`

Slide 17 — What if sum starts from 100?

Change `int sum = 0;` to `int sum = 100;`. Decide what the last line will be. Then look at the next slide.

Slide 18 — What if sum starts from 100?

The screen shows `Sum = 115`. The loop still adds 15. The start value is where the till begins.

Try it yourself. Put the 0 back. Change the program to add the numbers from 1 to 100. Only one number changes. (Expect `Sum = 5050`. Take out the `printf` inside the loop first, or you get 100 lines.)

Slide 19 — Common mistake: resetting inside the loop

This is the file `05_bug_sum_reset.c`. There is one new line inside the loop: `sum = 0;`. Decide what it prints. Then look at the next slide.

Slide 20 — Common mistake: the total is lost

The screen shows `Sum = 5`. Every round, `sum` is set back to 0. Then `i` is added. The till forgets everything. After the last round, `sum` holds only the last `i`, which is 5.

The compiler says **nothing**. No error, no warning. The program runs, and the answer is wrong. You must find this bug with your eyes.

Slide 21 — Common mistake: making sum inside the loop

This is the file `06_bug_sum_inside.c`. Here `int sum = 0;` is inside the loop. This program does not compile. The compiler says `error: 'sum' undeclared`, and it points at the last `printf`. A box that is made inside `{ }` only lives inside those `{ }`. After the closing brace it is gone.

Rule: make the running total and set it to zero before the loop.

Slide 22 — More totals: you choose where it stops

The file `02_sum_to_n.c`. The same pattern, but you type where the loop stops. You type `100`. The screen shows `Sum = 5050`. You type `5`: `Sum = 15`.

Slide 23 — More totals: add until the user types 0

The file `03_sum_until_zero.c`. It adds the numbers that you type, until you type 0. We do not know how many numbers are coming, so it is a `while` loop. You type `5 10 -3 8 0`. The screen shows `Sum = 20`.

Look at the two `scanf` lines. One is before the loop. One is at the end of the body. You will see this shape again on slide 36.

There is one more file with a total: `04_average.c`. It reads n numbers and prints their sum and their average. You type `4`, then `10 20 30 5`. The screen shows `Sum = 65` and `Average = 16.25`.

Slide 24 — Counter pattern

Part 2: add one, but only when something happens.

Slide 25 — Counter pattern: counting events

A **counter** is the little brother of the running total. You do not add the value. You add **one**, and only when something happens. How many students passed? How many numbers are positive?

Slide 26 — The program: count.c

This is the file `07_count_positives.c`.

- `scanf("%d", &n);` you say how many numbers are coming.
- `for (int i = 0; i < n; i++)` the loop runs `n` times. (It starts at 0 and uses `<`. Slide 45 explains why that is `n` times.)
- `scanf("%d", &value);` read one number. The box `value` is used again every round. The old number is replaced.

Slide 27 — A counter starts at 0

`int count = 0;` Zero, because nothing has been counted yet. It is **before** the loop, like the running total.

Slide 28 — An if inside a loop

`if (value > 0) { count++; }` An `if` **inside** a loop. The question is asked again every round, for the new number. `count` grows only when the answer is yes.

Slide 29 — How many positive values?

You type `5`, Enter, then `3 -1 0 7 -4`, Enter. Decide what the program prints. Then look at the next slide.

Slide 30 — How many positive values?

The screen shows `Positive values: 2`.

Round	value	value > 0?	count after
1	3	yes	1
2	-1	no	1
3	0	no	1
4	7	yes	2
5	-4	no	2

Look at round 3. **Zero is not positive.** `0 > 0` is false.

Run it again. You type `3`, then `-2 -5 0`. The screen shows `Positive values: 0`. That is why `count` starts at 0. If nothing is counted, zero is the right answer.

Slide 31 — Count without input

The file `08_count_multiples_of_3.c`. `i` goes from 1 to 20. `i % 3 == 0` means "the remainder after dividing `i` by 3 is 0", so `i` can be divided by 3. Decide what it prints. Then look at the next slide.

Slide 32 — Count without input

The screen shows `Count: 6`. The six numbers are 3, 6, 9, 12, 15, 18.

Slide 33 — A cousin: the largest value so far

The file `09_largest.c`. One box, `largest`, remembers the largest number seen so far. It starts as the first number that you type. Every round: is the new number larger? Then remember the new number.

You type `5`, then `3 9 -2 14 6`. The screen shows `Largest: 14`. You type `3`, then `-5 -2 -9`. The screen shows `Largest: -2`. That is why `largest` does not start at 0: with only negative numbers, 0 would be a wrong answer.

Slide 34 — Checking input with `while`

Part 3: ask again until the input is good.

Slide 35 — Input validation with while

People type nonsense. An age of `-5`. An age of `200`. A good program does not calculate with nonsense. It asks again. How many times? You do not know. Maybe never, maybe six times. **When you do not know how many rounds you need, use `while`.**

Slide 36 — The program: age.c

This is the file `10_age_check.c`. The next four slides go through it step by step.

Slide 37 — Step 1: ask once, before the loop

First, ask once and read once. This is **before** the loop.

Slide 38 — Step 2: is the input bad?

`while (age < 0 || age > 120)` Read it aloud: "while the age is below 0 **or** above 120". The `||` is "or", from last week. The condition describes the **bad** input.

Slide 39 — Step 3: ask again

Inside the loop: say it is invalid, and read again. The `scanf` inside the loop **is** the update. It is the only line that can make the condition false. If you forget it, the age never changes, and the program prints `Invalid. Try again:` for ever. Press Ctrl+C.

Part	Where it is
Start	the first `scanf`, before the loop
Check	the `while` condition: is the age below 0 or above 120?
Body	the `Invalid` message
Update	the second `scanf`, inside the loop

Slide 40 — Step 4: after the loop, trust the input

After the loop, you **know** the age is good. Every line after the loop can trust it.

Slide 41 — Run it

You type `-5`, Enter. Then `200`, Enter. Then `19`, Enter. The screen shows:

```
Enter age: -5
```

```
Invalid. Try again: 200
Invalid. Try again: 19
Accepted: 19
```

For now, type only numbers. If you type a letter, this program goes round for ever. Press Ctrl+C. Why that happens comes later in the course.

Slide 42 — Test the edges

You type	What happens	Why
`0`	`Accepted: 0` at once	`0 < 0` is false and `0 > 120` is false
`120`	`Accepted: 120` at once	`120 > 120` is false
`121`	`Invalid. Try again:`	`121 > 120` is true
`-1`	`Invalid. Try again:`	`-1 < 0` is true

For `0` and `120` the body runs **zero times**. That is fine. A `while` loop checks first, so it can run zero times.

Slide 43 — Same shape, new condition

The file `11_month_check.c` asks for a month and accepts only 1 to 12. It is the same program with a new condition: `month < 1 || month > 12`. You type `0`, then `13`, then `7`. The screen shows `Invalid. Try again:` twice, and then `Month 7 accepted`.

Try it yourself. Change it to ask for a mark from 0 to 100.

Slide 44 — Choosing loop boundaries

Part 4: `<` or `<=`? Start at 0 or at 1?

Slide 45 — Choosing loop boundaries

Both loops run five times. The left one shows `0 1 2 3 4`. The right one shows `1 2 3 4 5`.

Two styles, both correct.

- **1 to n, with `<=`.** This is how people count. Good for "print the numbers 1 to 10".
- **0 to n − 1, with `<`.** This is how C counts. `for (int i = 0; i < n; i++)` runs exactly `n` times. You will need this style for arrays in a few weeks.

Slide 46 — Which loop prints exactly 10 numbers?

The mistakes come from mixing the two styles. Decide for each of the four loop heads: how many numbers does it print? Then look at the next slide. (The file `14_boundaries.c` runs all four.)

Slide 47 — Which loop prints exactly 10 numbers?

Loop head	Screen shows	How many numbers
`i = 0; i < 10`	`0 1 2 3 4 5 6 7 8 9`	10
`i = 1; i <= 10`	`1 2 3 4 5 6 7 8 9 10`	10
`i = 0; i <= 10`	`0 1 2 3 4 5 6 7 8 9 10`	11
`i = 1; i < 10`	`1 2 3 4 5 6 7 8 9`	9

The first and the second print exactly 10 numbers. The third prints one too many. The fourth prints one too few. This mistake has a name: the **off-by-one error**. It is the most common loop bug in the world.

Slide 48 — Do not choose < or <= by feeling

Ask two questions:

1. What is the **first** value I want?
2. What is the **last** value I want?

Then test the last value by hand. Example: I want 10 to be included. Is `10 <= 10` true? Yes. So `i <= 10` is right. Is `10 < 10` true? No. So `i < 10` would stop at 9.

Try it yourself. Write a loop that prints `10 9 8 7 6 5 4 3 2 1`. (Start at 10. The check is `i >= 1`. The update is `i--`.)

Slide 49 — `break` and `continue`

Part 5: two words that change the normal flow of a loop.

Slide 50 — break: what prints?

The file `15_break.c`. **`break` means: leave the loop now.** You know it from `switch`. Decide what the screen shows. Then look at the next slide.

Slide 51 — break: leave the loop now

The screen shows `1 2 3 4 5`. When `i` is 6, the `break` runs **before** the `printf`. So 6 is not printed, and 7, 8, 9, 10 never happen.

Slide 52 — continue: what prints?

The file `16_continue.c`. **`continue` means: skip the rest of this round, and go on with the next round.** Decide what the screen shows. Then look at the next slide.

Slide 53 — continue: skip the rest of this round

The screen shows `1 2 4 5`. When `i` is 3, the `printf` is skipped. Then the loop carries on with 4. (After `continue`, the update `i++` still runs. That is why the loop does not get stuck on 3.)

Slide 54 — break and continue

Word	What it does
`break`	out of the loop, for good
`continue`	skip to the next round

Use them carefully. Most loops need neither. The age check has no `break`. Its condition does the work. If you need `break` because your loop condition is wrong, fix the condition.

Slide 55 — Two more examples

Left, the file `17_break_squares.c`: the screen shows `1 4 9 16 25`. When `i` is 6, `i * i` is 36. That is more than 30, so the loop ends.

Right, the file `18_continue_odds.c`: the screen shows `1 3 5 7 9`. Every even number is skipped.

Slide 56 — A real use of break: three tries for a PIN

The file `13_pin_three_tries.c`. Like a bank card: you get only three tries. The `for` loop counts the tries. If the PIN is right, no more tries are needed, so `break`. After the loop, the box `ok` says **why** the loop ended: 1 means the PIN was right, 0 means the tries ran out.

You type `1111`, then `1234`. The screen shows `Welcome`. You type three wrong PINs. The screen shows `Card blocked`.

Slide 57 — Worked examples

Part 6: two whole programs with a loop inside.

Slide 58 — Worked example: multiplication table

The file `19_table.c`. It reads `n`. Then `i` goes from 1 to 10, and every round prints one line.

Slide 59 — Same code, different value

You type `7`. The screen shows ten lines, from `7 x 1 = 7` to `7 x 10 = 70`.

The `printf` has three holes, `%d x %d = %d`, and three values: `n`, `i`, `n * i`. The third value is a calculation, done right inside the `printf`.

Slide 60 — One line of code, ten lines on the screen

`n` stays 7 in every line. `i` changes: 1, 2, 3 and so on up to 10. The body is **one** line of code. It prints **ten** different lines, because `i` is different every round. That is the idea of a loop: same code, different value.

Try it yourself. Make the table go up to 12. Only one number changes. (The `10` in `i <= 10` becomes `12`. The last line for `7` is then `7 x 12 = 84`.)

Slide 61 — A look ahead: a loop inside a loop

The file `20_table_grid.c`. **You do not need this for Quiz 2.** A loop can be inside another loop. The outer loop makes the rows. For **every** row, the inner loop runs five times and makes the columns. `%4d` prints a number in a space 4 characters wide, so the columns line up. This comes back in October.

Slide 62 — Worked example: factorial

5 factorial, written 5!, is 1 × 2 × 3 × 4 × 5 = 120. This is the running total again, but with **times** and not **plus**. A running product.

Slide 63 — Worked example: factorial

The file `21_factorial.c`. The two bright lines are the pattern: `int fact = 1;` before the loop, and `fact *= i;` inside it. `fact *= i` means `fact = fact * i`. The screen shows `120`.

Slide 64 — Factorial, round by round

Round 1: 1 × 1 = 1. Round 2: 1 × 2 = 2.

Slide 65 — Factorial, round by round

Round	`i`	`fact` before	`fact *= i`	`fact` after
1	1	1	1 × 1	1
2	2	1	1 × 2	2

Round	i	fact before	fact *= i	fact after
3	3	2	2 × 3	6
4	4	6	6 × 4	24
5	5	24	24 × 5	120

Slide 66 — Why does fact start at 1?

The sum started at 0. Why does `fact` start at 1? Decide what the program prints with `int fact = 0;`. Then look at the next slide.

Slide 67 — If fact starts at 0

The file `28_bug_fact_zero.c`. The screen shows `0`. Anything times 0 is 0. If `fact` starts at 0, it stays 0 for ever. Multiplying by 1 changes nothing, just as adding 0 changes nothing.

Rule: when you add, start at 0. When you multiply, start at 1.

Slide 68 — Mini-task: read n from the user

The file `22_factorial_n.c`. Two lines are new: `int n;` without a value, and a `scanf` that reads it.

Slide 69 — Now watch the numbers grow

You type	Screen shows	Correct?
5	5! = 120	yes
10	10! = 3628800	yes
12	12! = 479001600	yes
13	13! = 1932053504	no
17	17! = -288522240	no

What happened at 13? The real value of 13! is 6227020800. The program printed something else. There was no error and no warning, even with `-Wall`.

An `int` box has a size. The biggest number it can hold is 2147483647, about two thousand million. 13! is bigger than that. It does not fit, and C does not tell you. It keeps what fits, and that is a wrong number. For 17 the wrong number is even negative.

You do not need to fix this today. You need to know it exists. When a loop multiplies, numbers grow very fast. If a result looks crazy, or turns negative, remember this slide.

Slide 70 — Common Week 3 bugs

Part 7: the bugs everybody makes. Each one is small, and each one changes the whole result.

Slide 71 — Common Week 3 bugs

1. **Missing update.** The condition never becomes false, so the loop never stops. Press Ctrl+C, then look for the line that should change the variable. (File `26_bug_missing_update.c`.)
2. **Wrong comparison.** `<` instead of `<=`, or the other way round. One round too many or one too few. See slide 47. (File `27_bug_off_by_one.c`.)
3. **Resetting inside the loop.** The total is lost every round. See slides 20 and 21.
4. **A semicolon after the loop head.** See slide 73.
5. **`=` instead of `==`.** See slide 74.

6. **Bad starting value.** `i` starts from the wrong number (0 when you wanted 1), or `fact` starts at 0. See slide 67.

Slide 72 — The semicolon bug

The file `23_bug_semicolon.c`. There is a semicolon at the end of the `for` line. Decide what the screen shows. Then look at the next slide.

Slide 73 — The semicolon bug: it prints 6, once

You expect `1` to `5`. The screen shows only `6`, only once.

For C, a semicolon on its own is a complete statement that does nothing. So the body of this loop is "nothing". The loop runs five times doing nothing, and `i` climbs to 6. Then the block in `{ }` is just a normal block. It runs once and prints 6.

With `-Wall` the compiler warns you: `warning: this 'for' clause does not guard...` In plain words: "that block does not belong to the `for`. It only looks as if it does." It is a warning, so the program still builds. Read your warnings.

Rule: no semicolon after `for (...)` or `while (...)`.

Slide 74 — = instead of == inside a loop

The file `25_bug_assign_in_if.c`. `i = 3` is not a question. It puts 3 into `i`. Every round. So `i` never gets past 3, the check `i <= 5` is always true, and `three` prints for ever. Press Ctrl+C. The compiler warns: `suggest parentheses around assignment used as truth value`. **Inside a condition, always two equals signs.**

Slide 75 — Two loops that never stop

Left, the file `26_bug_missing_update.c`: the update `i++;` is missing. The program prints `1` for ever. The compiler gives **no** warning.

Right, the file `24_bug_while_semicolon.c`: a semicolon after `while (x <= 5)`. The body is empty, `x` never changes, and the program hangs with **nothing** on the screen. The compiler warns about this one.

For both: click in the terminal and press Ctrl+C.

Slide 76 — The debugging habit

Before you run a loop, put your finger on its four parts: start, check, body, update. If you cannot find one of them, that is your bug.

Slide 77 — Code tracing exercise

Part 8. On the quiz you cannot run the code. You trace it with a pen. **A question like this one will be on Quiz 2.**

Slide 78 — Code tracing exercise

The file `29_trace_xy.c`. Try it on paper before you read on.

How to trace a loop. Draw a table. One column for each variable. One row for each round. In every round, first do the check. If it is true, do the lines of the body **in order** and write the new values.

Slide 79 — Tracing: round 1

`2 < 8` is true. `y` is 0 + 2 = 2. Then `x` is 2 + 2 = 4.

Slide 80 — Tracing: round 2

`4 < 8` is true. `y` is 2 + 4 = 6. Then `x` is 6.

Slide 81 — Tracing: round 3

`6 < 8` is true. `y` is 6 + 6 = 12. Then `x` is 8. A classic mistake is to stop one round early, because 6 "feels" close to 8. The loop does not feel. `6 < 8` is true, so it goes again.

Slide 82 — Tracing: the loop ends

Round	Check `x < 8`	`y += x`	`x += 2`
before the loop		`y` is 0	`x` is 2
1	`2 < 8` true	`y` is 0 + 2 = 2	`x` is 4
2	`4 < 8` true	`y` is 2 + 4 = 6	`x` is 6
3	`6 < 8` true	`y` is 6 + 6 = 12	`x` is 8
4	`8 < 8` false	the loop ends	

The four answers.

1. After round 1: `x` is 4, `y` is 2. After round 2: `x` is 6, `y` is 6. After round 3: `x` is 8, `y` is 12.
2. The body runs **3** times.
3. The screen shows `x=8 y=12`.
4. `x += 2` moves the loop toward stopping. Without it, `x` stays 2 for ever.

The other classic mistake: running a fourth round, reading `<` as `<=`. `8 < 8` is false.

Notice the final `x`. It is 8, not 6. The loop ends **because** `x` became 8. After a loop, the variable holds the first value that failed the check.

Try it yourself. Trace on paper first, then run.

- Change the condition to `x <= 8`. (Now there is a fourth round. The screen shows `x=10 y=20`.)
- Put the condition back to `x < 8`. Start with `x = 1` and change the update to `x += 3`. (Rounds: `y` is 1, 5, 12 and `x` is 4, 7, 10. The screen shows `x=10 y=12`.)

Slide 83 — One more to trace at home

The file `30_trace_halving.c`. `n` is cut in half again and again. Remember: `int` divided by `int` throws the decimals away, so `5 / 2` is 2. Trace it on paper with a table. Then run the file to check your answer.

Slide 84 — Practice

Part 9: now you write.

Slide 85 — In-class practice tasks

Six small programs. Tasks 2 and 5 were written in class. Tasks 1, 3, 4 and 6 are for the weekend. **Before you type, write in plain words:** what goes in, what comes out, where the loop starts, when it stops, and what the update is.

Task 1. Print the numbers from 1 to n. Example run: you type `5`, the screen shows `1 2 3 4 5`. Hint: read `n` with `scanf`. Then `for`, starting at 1, with `i <= n`.

Task 3. Read n values and print their sum. Example run: you type `3`, Enter, then `5 -2 8`, Enter. The screen shows `Sum = 11`. You type `4`, then `10 20 30 5`, it shows `Sum = 65`. Hint: read `n`. Loop `n` times. Each round, read one `value` and do `sum += value;`. `sum` starts at 0, before the loop.

Task 4. Read n values and count the positive ones. Example run: you type `3`, then `5 -2 8`. The screen shows `Positive values: 2`. Hint: this is slide 26. Close the slides and write it from memory. Then compare.

Task 6. Print a multiplication table for one number. Example run: you type `3`, the screen shows ten lines, from `3 x 1 = 3` to `3 x 10 = 30`. Hint: this is slide 58. Close the slides and write it from memory.

Before you say a program is done: it compiles with `-Wall` and no warnings. You tested it with at least two inputs. For a loop, also test the edges: the smallest `n` that makes sense, and the last value (is 10 printed when you type 10?). The code is indented.

Slide 86 — Task 2: even numbers from 2 to n

Read `n`. Print the even numbers from 2 to `n`. You type `10`, the screen shows `2 4 6 8 10`. You type `9`, it shows `2 4 6 8`.

Slide 87 — Task 2: two good ways

The first way (file `31_evens_step.c`): start at 2 and jump by two. `i += 2` adds 2 after every round.

The second way (file `32_evens_if.c`): visit every number from 1 to `n`, and print it only if it is even. `i % 2 == 0` means "the remainder after dividing by 2 is 0", so `i` is even.

Both are correct.

Slide 88 — Task 5: keep asking for the PIN

Keep asking for a password until it is correct. You cannot read words yet, so the password is a whole number, a PIN: `1234`. The box on the slide shows what the screen should look like when you type `1111`, then `4321`, then `1234`.

Slide 89 — Task 5: it is the age check again

The file `12_pin.c`. It is the age check with a different condition. `pin != 1234` means "the PIN is not 1234". That is the **bad** input, so the loop goes on while it is true.

If that was easy: give the user only three tries. Try it yourself first. Then look at `13_pin_three_tries.c`.

Slide 90 — Quiz-style question 1

Try it on paper first.

`1 2 3`. Round 1: `1 <= 3` true, print 1, `i` is 2. Round 2: print 2, `i` is 3. Round 3: `3 <= 3` true, print 3, `i` is 4. Then `4 <= 3` is false. The loop ends.

Slide 91 — Quiz-style question 2

Try it on paper first. Then look at the next slide.

Slide 92 — Quiz-style question 2: the answer

`5 6`. `x++` means: use the old value first, then add one. So the first `printf` gets 5. After that, `x` is 6, and the second `printf` shows it. With `++x` in the first line, the output would be `6 6`.

Slide 93 — Quiz-style question 3

1. The rounds: 0 + 1 = 1, 1 + 2 = 3, 3 + 3 = 6, 6 + 4 = 10. If the line were `p *= i` with `p` starting at 1, the answer would be 24.

Why show the trace? If your final number is wrong but your table is right up to the last row, the table earns marks. A bare wrong number earns nothing.

The three questions are one program in the file `33_quiz_style.c`. Run it to check.

Slide 94 — Week 3 summary

After this week you can:

- Write a `while` loop and a `for` loop, and choose between them: `for` when you know how many rounds, `while` when you do not.
- Use `++`, `--`, `+=`, `-=`, `*=` to update a variable.
- Keep a running total (start at 0, `+=`) or a running product (start at 1, `*=`), set up **before** the loop.
- Count how often something happens, with an `if` inside the loop.
- Keep asking until the input is good.
- Choose `<` or `<=` on purpose, by checking the first and the last value.
- Say what `break` and `continue` do.
- Trace a loop with a table, one row per round.

Slide 95 — Always ask, for every loop

Where does it start? When does it stop? What changes each time? What should be printed or stored?

Slide 96 — Before Tuesday

1. **Do Knowledge check 6** on Blackboard: this class's slides. Six questions, three attempts, best kept, worth 0%. Knowledge check 5 covers Tuesday's slides and is open too.
2. **Write the four programs** left on slide 85: Tasks 1, 3, 4 and 6. Bring them Tuesday, working or not.
3. **Quiz 2 is on Friday 2 October**, in the first 25 minutes of class, on paper, no laptop. It covers data types, conditionals and loops: classes 3 to 6. Same shape as Quiz 1: read a program and say what it prints, then write one. Practise tracing with a pen, like slide 78.
4. **Play with the code.** Blackboard, Week 3 module, *Class 6 code*. Every file has a **Try** list at the top. Do the changes and see what happens.

Next week: functions. Giving a name to a piece of program, so you can use it again.